Universidad de Extremadura

Escuela Politécnica

Ingeniería Informática en Ingeniería del Software

Trabajo Fin de Grado

Optimización de las emisiones de CO2 en redes definidas por software mediante algoritmos bioinspirados

Santiago Rodríguez-Arias Martínez-Berná

Convocatoria, 2026

Universidad de Extremadura

Escuela Politécnica

Ingeniería Informática en Ingeniería del Software

Trabajo Fin de Grado

Optimización de las emisiones de CO2 en redes definidas por software mediante algoritmos bioinspirados

Autor: Santiago Rodríguez-Arias Martínez-Berná

Tutor: Jaime Galán Jiménez

Co-Tutor: José Antonio Gómez de la Hiz

Resumen

A día de hoy, las redes de comunicaciones son un componente esencial en la vida diaria de las personas y en el funcionamiento de las empresas. El aumento del tráfico de datos ha crecido de forma exponencial, llegando a multiplicarse por 80 desde 2007 hasta 2023. Aunque el crecimiento del consumo energético no es proporcional al aumento del tráfico de datos—pues este último apenas se ha duplicado—, esto no significa que no haya un impacto ambiental significativo. Se estima que las redes de comunicaciones representan aproximadamente el 1 % del consumo eléctrico mundial y alrededor del 1,4 % de las emisiones globales de CO_2. Y estos datos no van a hacer más que aumentar debido al auge de las Inteligencias Artificiales y los grandes centros de procesamiento de datos. Por tanto, es fundamental buscar soluciones que permitan optimizar el uso de las redes para reducir su impacto ambiental.

En este trabajo se propone una solución basada en redes definidas por software y algoritmos bioinspirados para optimizar las emisiones de CO_2 asociadas al uso de las redes de comunicaciones. Más concretamente, se hace uso de la investigación realizada en [1], que proporciona una fórmula para calcular las emisiones de CO_2 en función del tráfico de datos y el consumo energético. Con esta fórmula en mente, se ha modelado una función de optimización que se usará en un algoritmo de optimización por enjambre de partículas (PSO, por sus siglas en inglés) para encontrar la configuración de red que minimice las emisiones de CO_2. Se han usado diferentes topologías de red y matrices de tráfico para evaluar la viabilidad del algoritmo en diferentes escenarios.

Se presentan dos versiones diferentes del algoritmo, así como diferentes maneras de realizar el enrutado

Palabras clave: PSO, redes cableadas, enrutamiento verde, eficiencia energética.

Abstract

EL RESUMEN EN INGLÉS

Keywords: Most important or descriptive words, separated by commas.

Índice general

1. Introducción **1**

1.1. Secciones 1

1.1.1. una subsección 1

1.1.2. otra subsección 1

1.2. Citas y referencias 2

1.3. Imágenes 2

1.4. Código 3

2. Objetivos **4**

3. Estado del Arte **5**

4. Metodología **6**

5. Implementación y desarrollo **7**

5.1. Elección de la técnica de optimización 7

5.2. Particle Swarm Optimization 8

5.3. Parámetros de PSO 9

5.3.1. Partícula 10

5.3.2. Función objetivo 10

5.3.3. Velocidad 10

5.3.4. Posición 11

5.3.5. Coeficiente de inercia 11

5.3.6. Coeficiente cognitivo 11

5.3.7. Coeficiente social 12
5.3.8. Número de vecinos 12
5.3.9. Dimensiones del espacio de búsqueda 12
5.4. Aproximación al problema 13
5.5. Entorno de desarrollo 14
5.5.1. Módulo discreto de PySwarms 14
5.6. Representación de partículas 15
5.6.1. Variables de decisión 15
5.6.2. Relación entre las partículas y la red 16
5.7. Función objetivo y enrutado 16
5.8. Manejo de restricciones 18
5.8.1. PSO normal 18
5.8.2. PSO VCH 18
5.9. Integración con librerías 18

6. Resultados **19**

7. Conclusiones y trabajos futuros **20**

Anexos **21**

A. Ejemplo de anexo **22**
A.1. Sección de anexo 22

Bibliografía **23**

Índice de tablas

Índice de figuras

1.1. Logo EPCC 2

5.1. Diagrama de flujo de PSO 9

5.2. Función sigmoide 13

Capítulo 1

Introducción

Esta plantilla sirve como ejemplo de TFE. Las secciones son las que están en la normativa, aunque si el tutor lo considera se puede cambiar la estructura. Esta sección la aprovechamos para introducir brevemente LaTeX.

1.1. Secciones

Las secciones se crean con *section*. Las subsecciones y subsubsecciones con *subsection* y *subsubsection*, respectivamente. Si se desea que alguna subsección en concreto no salga en el índice se pueden usar los mismos comandos añadiéndoles un asterisco al final (*subsection** y *subsubsection**).

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna.

1.1.1. una subsección

Lorem ipsum dolor sit amet, consectetuer adipiscing elit.

1.1.2. otra subsección

Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis.

subsubsecciones

¡Por defecto las subsubsecciones no aparecen en el índice! Este comportamiento se puede cambiar, pero lo recomendable es organizar los capítulos en 3 niveles para facilitar la lectura.

1.2. Citas y referencias

Para citar un texto hay que incluirlo en el fichero *LocalBibliography.bib* de esta plantilla. Una vez hecho se puede referenciar usando el comando *cite* [**?**]. Para referenciar imágenes, secciones o tablas se usa el comando *autoref*. Por ejemplo Figura 1.1 o Capítulo 2.

Para facilitar la lectura, cuando se insertan citas y referencias es conveniente insertar un carácter de espacio sin salto de línea ˜ antes del comando de cita o referencia.

1.3. Imágenes

Las imágenes se insertan así:

Figura 1.1: Logo EPCC.
Fuente: https://politecnica.unex.es

1.4. Código

El código se puede insertar así:

```
echo "Este es un ejemplo de cómo se inserta código con la plantilla de Trabajo
↪  Fin de Estudios de la Escuela Politécnica de Cáceres"
```

También se puede seleccionar desde un fichero, por ejemplo:

```
#include <iostream>
using namespace std;

int main() {
  cout << "Hello World!";
  return 0;
}
```

Los lenguajes reconocidos para marcar se pueden consultar en https://pygments.org/docs/lexers/. Aunque depende del tipo de trabajo, se recomienda incluir código sólo en anexos.

Capítulo 2

Objetivos

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna.

Capítulo 3

Estado del Arte

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna.

Capítulo 4

Metodología

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna.

Capítulo 5

Implementación y desarrollo

En esta sección se presenta detalladamente la implementación del algoritmo desarrollado para abordar el objetivo de este trabajo, cómo se ha planteado este algoritmo, sus atributos y los distintos enfoques que se han probado hasta llegar a la solución actual.

5.1. Elección de la técnica de optimización

Como se ha visto en el apartado del estado del arte, los problemas de optimización se pueden abordar con diferentes técnicas dependiendo de las características del problema.

Los algoritmos genéticos pueden ser una buena opción, pero suelen requerir una mayor cantidad de tiempo de ejecución, especialmente para problemas con un gran espacio de búsqueda. Al estar tratando con un escenario en el que el tiempo es crítico, pues las necesidades de la red cambian a lo largo del día, este tipo de algoritmos no es la mejor opción.

Por otro lado, los algoritmos voraces son rápidos y fáciles de implementar, pero son "miopes", es decir, toman decisiones basadas únicamente en la información local sin considerar el impacto a largo plazo. Esto puede llevar a soluciones subóptimas o, incluso, a soluciones inviables si se toman decisiones que comprometen la conectividad de la red.

En cambio, los algoritmos de PSO son una gran opción para este tipo de problemas, ya que son capaces de explorar el espacio de búsqueda de manera eficiente y encontrar

soluciones, aunque no sean óptimas, en un tiempo razonable. Además, como PSO es un algoritmo de optimización global, tiene la capacidad de evitar caer en óptimos locales. Teniendo en cuenta que se tratan de problemas con grandes espacios de búsqueda, es fundamental evitar a toda costa los óptimos locales.

Otra ventaja de PSO es que existen numerosas implementaciones disponibles, lo que facilita su uso y permite centrarse en la modelización de la función objetivo. Además, PSO es un algoritmo relativamente sencillo de ajustar, con pocos parámetros a configurar, facilita la realización de barridos para encontrar la configuración que mejor se adapte al problema en cuestión.

Es por esto que se ha optado por usar PSO como técnica de optimización para abordar el problema de optimización de las emisiones de CO_2 en redes.

5.2. Particle Swarm Optimization

Como en la mayoría de algoritmos de optimización, PSO se basa en la idea de iterar sobre un conjunto de soluciones candidatas —partículas— y actualizar estas soluciones en función de su rendimiento. Cada partícula tiene una posición en el espacio de búsqueda, que representa una posible solución al problema, y una velocidad que determina cómo se mueve esa partícula a lo largo del espacio de búsqueda. En cada iteración, cada partícula actualiza su velocidad y posición en función de su mejor posición personal y la mejor posición global encontrada por el enjambre. Una vez alcanza la condición de parada, como puede ser un número máximo de iteraciones, el algoritmo devuelve la mejor solución encontrada.

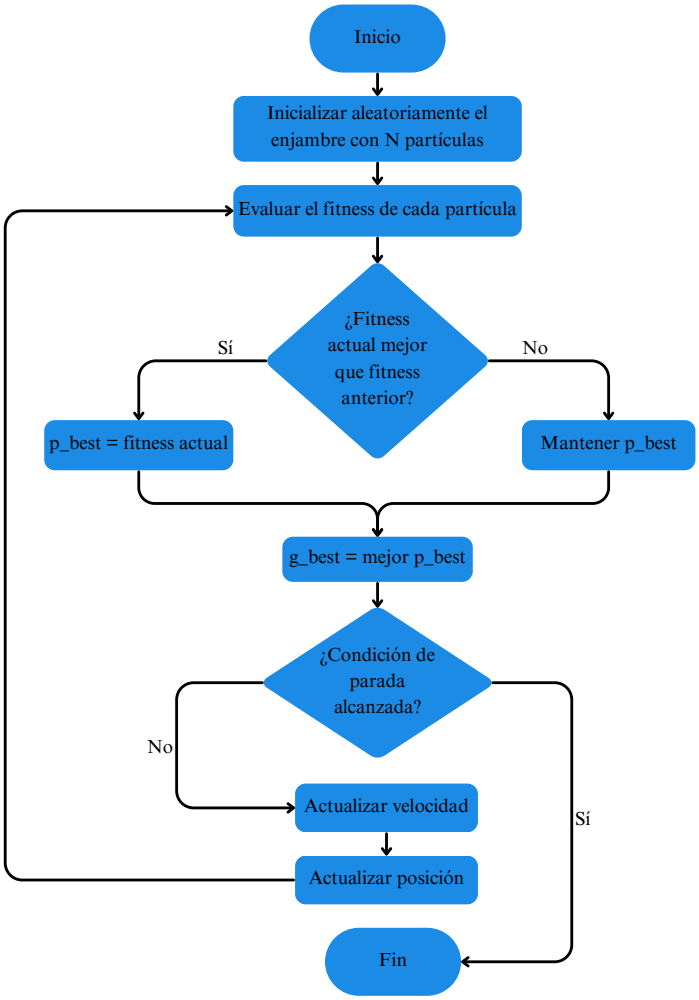

Figura 5.1: Diagrama de flujo de PSO.
Fuente: Elaboración propia.

5.3. Parámetros de PSO

Con lo que se ha explicado hasta ahora, se han mencionado algunos de los parámetros de PSO, como pueden ser el número de partículas o el número de iteraciones. Sin embargo, existen otros parámetros que también son importantes a la hora de configurar el algoritmo. Estos parámetros son: coeficiente de inercia, coeficiente cognitivo, coeficiente social, número de vecinos y dimensiones del espacio de búsqueda. Cada uno de estos parámetros tiene un impacto significativo en el rendimiento del algoritmo y en la calidad de las soluciones encontradas, por lo que es importante entender su función y cómo afectan

al comportamiento del algoritmo.

5.3.1. Partícula

El concepto de partícula ya se ha comentado brevemente con anterioridad, y es que una partícula es un agente individual que representa una solución candidata al problema de optimización. Dado que PSO es un algoritmo inspirado en el comportamiento social de los animales, como las bandadas de pájaros o los bancos de peces, se puede imaginar cada partícula como un pájaro que está sobrevolando el terreno —espacio de búsqueda— en busca de la zona con más comida —mejor solución—.

5.3.2. Función objetivo

La función objetivo es una parte fundamental de cualquier algoritmo de optimización, pues se trata de la función que se usa para evaluar cada solución candidata y determinar qué tan buena es esa solución. En el caso de PSO, la función objetivo se evalúa para cada partícula en cada iteración, y se utiliza para actualizar la mejor posición personal de cada partícula y la mejor posición global del enjambre.

5.3.3. Velocidad

En PSO, las partículas tienen dos atributos principales, siendo la velocidad uno de ellos. Esta determina cómo se mueve cada partícula a lo largo del espacio de búsqueda. La velocidad se actualiza en cada iteración en función de la mejor posición personal de la partícula y la mejor posición global del enjambre, lo que permite que las partículas exploren el espacio de búsqueda de manera eficiente. Una velocidad alta puede permitir que las partículas exploren el espacio de búsqueda de manera más amplia, pero también puede hacer que el algoritmo sea más propenso a saltar sobre buenas soluciones. Por otro lado, una velocidad baja puede hacer que el algoritmo sea más lento, pero también puede ayudar a que las partículas se concentren en áreas prometedoras del espacio de búsqueda.

5.3.4. Posición

El segundo atributo principal de las partículas es la posición, que representa una posible solución al problema de optimización. La diferencia entre la posición y la partícula, pues las definiciones pueden ser un poco confusas, es que la posición es el valor actual de la solución representada por la partícula, mientras que la partícula es el agente que se mueve a lo largo del espacio de búsqueda. Siguiendo con el ejemplo de los pájaros, la posición sería el lugar específico donde se encuentra cada pájaro en un momento dado. La posición se actualiza en cada iteración en función de la velocidad de la partícula, lo que permite que las partículas se muevan a través del espacio de búsqueda y encuentre mejores soluciones.

5.3.5. Coeficiente de inercia

El coeficiente de inercia es un parámetro que controla la influencia de la velocidad anterior en la actualización de la velocidad actual de cada partícula. Un coeficiente de inercia alto hace que las partículas mantengan su velocidad anterior, lo que puede ayudar a que el algoritmo explore el espacio de búsqueda de manera más amplia. Por otro lado, un coeficiente de inercia bajo hace que las partículas sean más propensas a cambiar su velocidad en función de la mejor posición personal y global, lo que puede ayudar a que el algoritmo se concentre en áreas prometedoras. Es importante encontrar un valor adecuado para este parámetro en función del problema específico, ya que un valor demasiado alto o demasiado bajo puede afectar negativamente al rendimiento del algoritmo.

5.3.6. Coeficiente cognitivo

El coeficiente cognitivo es un parámetro que controla la influencia de la mejor posición personal de cada partícula en la actualización de su velocidad. Un coeficiente cognitivo alto hace que las partículas se centren más en su propia experiencia, lo que ayuda a que el algoritmo se centre en áreas prometedoras. Por otro lado, un coeficiente cognitivo bajo hace que las partículas sean más propensas a seguir la mejor posición global del enjambre,

lo que puede ayudar a que el algoritmo explore el espacio de búsqueda más ampliamente.

5.3.7. Coeficiente social

El coeficiente social es un parámetro que controla la influencia de la mejor posición global del enjambre en la actualización de la velocidad de cada partícula. Un coeficiente social alto hace que las partículas se centren más en la experiencia colectiva del enjambre, lo que puede ayudar a que el algoritmo explore el espacio de búsqueda más ampliamente. Por otro lado, un coeficiente social bajo hace que las partículas sean más propensas a centrarse en su propia experiencia, así el algoritmo se concentra en áreas prometedoras. Es importante encontrar un equilibrio adecuado entre el coeficiente cognitivo y social, ya que un desequilibrio puede desembocar en soluciones subóptimas.

5.3.8. Número de vecinos

Este parámetro controla la cantidad de partículas que se consideran como vecinos de cada partícula en la actualización de su velocidad. Si el número de vecinos es alto, cada partícula se ve influenciada por un gran número de otras partículas, lo cual hace que se centre más en la experiencia colectiva del enjambre. En cambio, si el número de vecinos es bajo, cada partícula se ve influenciada por un número reducido de otras partículas, centrándose más en su propia experiencia.

5.3.9. Dimensiones del espacio de búsqueda

Este parámetro define el número de dimensiones que tiene el espacio de búsqueda, lo que a su vez determina la cantidad de variables que se están optimizando. Por lo general, cuantas más dimensiones tenga el espacio de búsqueda, más complejo será el problema de optimización, ya que habrá más variables a considerar, pero puede permitir encontrar mejores soluciones. Por el contrario, un espacio de búsqueda con pocas dimensiones puede ser más fácil de explorar, pero también puede limitar la calidad de las soluciones encontradas.

5.4. Aproximación al problema

Con los conceptos básicos de PSO ya explicados, es importante entender cómo se ha aplicado este algoritmo al problema específico de optimización de las emisiones de CO_2 en redes.

La primera solución que se planteó fue tranformar el espacio discreto del problema en un espacio continuo, lo que permitiría usar PSO en su forma original. Sin embargo, esta aproximación terminó descartándose, pues la umbralización hacía que PSO trabajase a ciegas. Es decir, no era capaz de discernir entre los cambios que se producían por debajo del umbral —lo que se considera como un enlace apagado—, por tanto PSO no sabía si esos cambios apuntaban a una mejora o a un empeoramiento de la solución, lo que generaba un comportamiento errático del algoritmo.

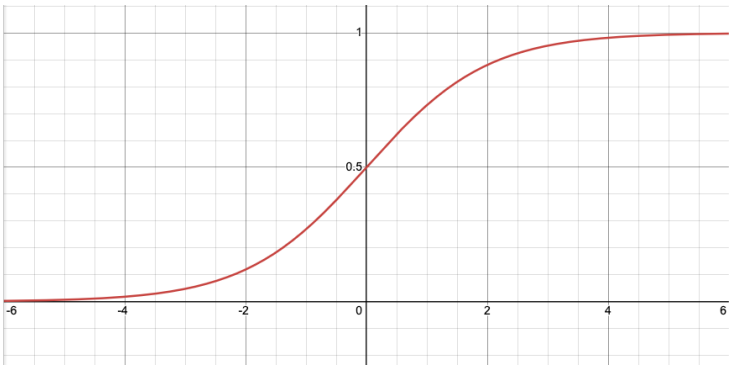

Figura 5.2: Representación gráfica de la función sigmoide.
Fuente: Elaboración propia.

Más adelante, se iteró sobre la idea anterior de transformar el espacio discreto en continuo, pero esta vez usando una función sigmoide para umbralizar a través del cálculo de la probabilidad de que un enlace esté activo o apagado. Sin embargo, y como se muestra en la Figura 5.2, se observó que, a medida que el valor continuo de la partícula —x— se aleja del cero, la curva se vuelve casi plana. Esto hace que, si el algoritmo decide que el enlace debe estar encendido, la velocidad de PSO empuje el valor de x hacia números positivos cada vez más altos. Pero si más tarde el algoritmo decide que ese enlace en realidad debe estar apagado, PSO tardará muchas iteraciones en generar una velocidad lo suficientemente negativa como para arrastrar el valor de x de vuelta al cero. En todo

ese tiempo, la partícula está estancada y pierde capacidad de exploración.

5.5. Entorno de desarrollo

Para la realización de este trabajo se ha utilizado el lenguaje de programación Python, debido a su versatilidad y a la gran cantidad de librerías disponibles para el desarrollo de algoritmos de optimización. Además, es el mismo lenguaje que se ha utilizado por parte del co-tutor, José Antonio Gómez de la Hiz, en sus investigaciones en el mismo ámbito, lo que ha sido de gran ayuda para la realización de este trabajo.

Inicialmente se valoraron dos herramientas para el desarrollo del algoritmo: Visual Studio Code y PyCharm. Finalmente se optó por PyCharm, puesto que ofrece diferentes funcionalidades que facilitan en gran medida el desarrollo de código, como pueden ser la integración de sistemas de control de versiones, la depuración de código o la gestión de entornos virtuales.

En cuanto a las librerías, se han utilizado principalmente las siguientes:

- **NumPy**: para el manejo de arrays y matrices, así como carga de datos desde archivos de texto o csv.
- **NetworkX**: para la creación y manipulación de grafos, fundamental para poder modelar las topologías de red usadas.
- **Matplotlib**: para la visualización de resultados y gráficos.
- **PySwarms**: para el uso de algoritmos de PSO en espacios dicretos.
- **SciPy**: para cálculos estadísticos adicionales, como los intervalos de confianza.

5.5.1. Módulo discreto de PySwarms

En el ámbito de las redes, el espacio de búsqueda por lo general es discreto, ya que las configuraciones de red suelen estar formadas por un conjunto finito de opciones, como pueden ser las rutas que se pueden usar para enrutar el tráfico o los enlaces que se pueden

activar o desactivar. Esto hace que sea necesario adaptar el algoritmo de PSO para que pueda trabajar en un espacio de búsqueda discreto.

Por suerte, PySwarms—que es una librería de optimización por enjambre de partículas en Python—, ofrece un módulo específico para trabajar con espacios de búsqueda discretos. Esto ha facilitado enormemente la implementación de la función de optimización, puesto que PySwarms se encarga de gestionar las operaciones de PSO, como la actualización de las posiciones y velocidades de las partículas.

5.6. Representación de partículas

En PSO, cada partícula representa una posible solución al problema de optimización. En este caso, cada partícula representa una configuración de red, la cual está formada por el conjunto de enlaces que están activos y apagados. Por tanto, cada partícula se representa como un vector binario, donde cada posición del array representa un enlace de la red, siendo uno (1) la representación de un enlace activo y cero (0) la representación de un enlace apagado.

5.6.1. Variables de decisión

El encendido y apagado de enlaces es la principal variable de decisión en este problema, ya que el objetivo es encontrar la configuración de red que minimice las emisiones de CO_2 y es la manera más directa de disminuir el consumo y las emisiones. Pero no es la única variable de decisión, ya que también se priorizan las rutas más cortas y eficientes para enrutar el tráfico.

Para ello, se aplica la estrategia *K-Shortest Paths* (KSP), que consiste en encontrar las K rutas más cortas entre cada par de nodos de la red. Al definir las emisiones de cada enlace como el peso de ese enlace, se consigue que las rutas más cortas sean también las rutas que menos emisiones generan. De esta manera, el algoritmo de PSO no solo busca apagar enlaces para reducir las emisiones, sino que también busca enrutar el tráfico por las rutas más eficientes en términos de emisiones.

También se barajó la posibilidad de apagar nodos enteros, pero se terminó descartando esta opción, pues apagar un nodo implica apagar todos los enlaces asociados a ese nodo, lo que puede dejar zonas totalmente desconectadas. Es por este motivo que se optó por apagar enlaces individuales, lo que ofrece mayor flexibilidad a la hora de buscar configuraciones de red que reduzcan las emisiones sin comprometer la conectividad de la red.

5.6.2. Relación entre las partículas y la red

Para poder evaluar la red en estudio y calcular la mejor configuración de red, es necesario establecer una relación entre las partículas y la red, concretamente, entre la matriz de adyacencia de la red y el vector binario que representa cada partícula. Para ello, se ha creado una función que toma como entrada la matriz de adyacencia de la red y devuelve una lista de enlaces, donde cada enlace se representa como una tupla de nodos *(src, dst)*. De esta manera, cada posición del vector binario de la partícula corresponde a un enlace específico de la red, y el valor de esa posición indica si ese enlace está activo o apagado. De esta manera, cuando PSO devuelve la mejor posición global, se puede traducir ese vector binario a una configuración de red concreta.

Definir esta relación entre las partículas y la red es fundamental por dos motivos principales. El primer motivo es que permite usar la función de optimización para evaluar las configuraciones de red de las partículas, ya que se necesita saber qué enlaces son los que están activos para evaluar si es una solución factible y calcular las emisiones asociadas. El segundo motivo es que permite interpretar los resultados e implementar la solución encontrada en la red real. Sin esta relación, sería difícil entender qué configuración de red corresponde a cada partícula y cómo aplicar la solución encontrada en la práctica.

5.7. Función objetivo y enrutado

La función objetivo es la función que se usa en PSO para evaluar cada partícula y determinar qué configuración de red es la mejor. En este caso, la función se ha modelado

en base a la fórmula de emisiones de CO_2 que se presenta en el artículo *Exploring the Benefits of Carbon-Aware Routing* [1]:

$$C_{tot}(G, X, \lambda, c, \beta) = \sum_{n=1}^{V}(x_n * \lambda_n * c_n) + \sum_{l=1}^{E}\beta_l(c_{l_{n_1}} + c_{l_{n_2}}) \tag{5.1}$$

Donde:

- G es la topología de la red, representada como un grafo.
- V y E son el número de nodos y enlaces de la red, respectivamente.
- x_n es la intensidad de flujo en el nodo n.
- c_n es la tasa de emisiones de CO_2 asociada al nodo n.
- λ_n es la potencia dinámica incremental por tasa de datos del nodo n.
- β_l es el incremento del consumo de energía al habilitar un puerto.
- $c_{l_{n_1}}$ y $c_{l_{n_2}}$ son las emisiones de CO_2 asociadas a los nodos n_1 y n_2 que están conectados por el enlace l.

El primer sumatorio representa el consumo dinámico de energía asociado al tráfico que pasa por cada nodo. Es decir, cuanto más tráfico pase por un nodo, más energía consumirá ese nodo, y por tanto, más emisiones de CO_2 generará. El segundo sumatorio se corresponde con el consumo estático de energía asociado a los enlaces que están activos. Un enlace activo implica que hay un puerto habilitado en cada uno de los nodos que conecta, lo que genera un consumo de energía adicional. Este segundo sumatorio es el que permite mayor optimización, pues al apagar enlaces—y por tanto, apagar puertos—se puede reducir significativamente el consumo de energía y las emisiones de CO_2.

Pero para poder evaluar una configuración de red concreta, no basta con calcular las emisiones asociadas a esa configuración, sino que también es necesario asegurarse que esa configuración permite enrutar todo el tráfico de la red sin generar congestión. Para ello, y como se ha mencionado con anterioridad en 5.6.1, se hace uso de la estrategia KSP

para enrutar el tráfico a través de las rutas más cortas y eficientes. Al tener varias rutas disponibles para cada par de nodos, en caso de que una ruta no pueda seguir enrutando tráfico, se pasa a la siguiente ruta más corta.

5.8. Manejo de restricciones

Hablar sobre las restricciones para que una solución sea factible.

5.8.1. PSO normal

Definir el algoritmo para PSO normal. Pros y contras.

5.8.2. PSO VCH

Definir el algoritmo para PSO VCH. Pros y contras.

5.9. Integración con librerías

Hablar sobre la integración con PySwarms y las diferentes opciones de configuración de PSO (número de partículas, iteraciones y paralelización).

Capítulo 6

Resultados

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna.

Capítulo 7

Conclusiones y trabajos futuros

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna.

Anexos

Anexo A

Ejemplo de anexo

Si no se desea incluir anexos, sólo hay que borrar este capítulo.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna.

A.1. Sección de anexo

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna.

Bibliografía

[1] Sawsan El-Zahr, Paul Gunning, and Noa Zilberman. Exploring the benefits of carbon-aware routing. *Proceedings of the ACM on Networking*, 1(CoNEXT3):1–24, 2023. doi:10.1145/3629165.